Cheatsheet: Trabalhando com DOM em JavaScript

Depuração JavaScript, Terminologias BOM e DOM	Descrição	Exemplo de Código

try{....} block	O código que pode gerar um erro está encapsulado dentro de um bloco try. Este bloco ajuda a monitorar erros.	`const obj = undefined;`
		`try {`
		`  const propertyValue = obj.property; // Attempting to access a property of an undefined o`
		`  console.log("Property Value: " + propertyValue);`
		`  console.log("This message will be reached.");`
		`} catch (error) {`
		`  console.error("An error occurred while accessing the property:", error.message);`
		`}`
		`console.log("Program continues after error handling.");`

bloco catch{....}	O bloco catch em JavaScript captura e trata erros que ocorrem dentro de um bloco try.	`try {`
		`  // Code that might throw an error`
		`  const result = nondeclaredFunction(); // Assuming someFunction() is not defined`
		`  console.log(result); // This line won't execute due to the error`
		`} catch (error) {`
		`  // Code to handle the error`
		`  console.log('An error occurred:', error.message);`
		`}`

Método getElementById()	getElementById é um método em JavaScript usado para acessar um elemento HTML específico dentro do Modelo de Objeto de Documento (DOM) com base em seu atributo id único.	`<!DOCTYPE html>`
		`<html>`
		`<head>`
		`  <title>getElementById Example</title>`
		`</head>`
		`<body>`
		`  <h1 id="main-heading">Welcome to the Example Page</h1>`
		`  <p id="content-paragraph">This is some content.</p>`
		`  <script>`
		`  const headingElement = document.getElementById('main-heading');`
		`console.log(headingElement)`
		`  </script>`
		`</body>`
		`</html>`

Método getElementsByClassName()	getElementsByClassName é um método em JavaScript que é usado para acessar múltiplos elementos HTML dentro do Modelo de Objeto de Documento (DOM) que compartilham o mesmo nome de classe.	<!DOCTYPE html>
		<html>
		<head>
		<title>getElementsByClassName Example</title>
		</head>
		<body>
		 This is a highlighted paragraph.
		 This is another highlighted paragraph.
		 This is a regular paragraph.
		<script>
		const highlightedElements = document.getElementsByClassName('highlighted');
		// Modify the text content of each element
		for (let i = 0; i < highlightedElements.length; i++) {
		highlightedElements[i].textContent = `This paragraph is highlighted! for class
		}
		</script>
		</body>
		</html>

Método getElementsByTagName()	getElementsByTagName é um método em JavaScript que é usado para acessar múltiplos elementos HTML dentro do Modelo de Objeto de Documento (DOM) com base no nome da sua tag.	<!DOCTYPE html>
		<html>
		<head>
		<title>getElementsByTagName Example</title>
		</head>
		<body>
		 Heading 2
		 This is a paragraph.
		 This is another paragraph.
		<script>
		const paragraphElements = document.getElementsByTagName('p');
		console.log(paragraphElements);
		console.log(paragraphElements[0]);
		console.log(paragraphElements[1]);
		</script>
		</body>
		</html>

querySelector	querySelector é um método usado para acessar elementos HTML dentro do Modelo de Objetos do Documento (DOM) com base em seletores semelhantes ao CSS, como classe, ID ou nome da tag.	<!DOCTYPE html>
		<html>
		<head>
		<title>querySelector Example</title>
		</head>
		<body>
		 This is a highlighted paragraph.
		 This is a paragraph with an ID.
		 This is a regular paragraph.
		<script>
		const elementByClass = document.querySelector('.highlighted');
		// Log the selected element to the console
		console.log(elementByClass);
		// Select the element with the ID "my-paragraph" using querySelector
		const elementByID = document.querySelector('#my-paragraph');
		// Log the selected element to the console
		console.log(elementByID);
		// Select the first element using querySelector
		const elementByTag = document.querySelector('div');
		// Log the selected element to the console
		console.log(elementByTag);
		</script>
		</body>
		</html>

querySelectorAll	querySelectorAll é um método usado para selecionar múltiplos elementos HTML com base em seletores semelhantes ao CSS,	<!DOCTYPE html> <html> <head> <title>querySelectorAll Example</title> </head> <body> This is a highlighted paragraph. This is a highlighted paragraph. This is another highlighted paragraph.

	como classe, ID ou nome da tag, e retorna uma coleção de elementos do tipo Node-List que correspondem ao seletor especificado.	<section>This is a regular paragraph.</section>
		<script>
		const elementsById = document.querySelectorAll('#highlight');
		const elementsByClass = document.querySelectorAll('.highlighted');
		const elementsByTag = document.querySelectorAll('section');
		// Log the selected elements to the console
		console.log(elementsById);
		console.log(elementsByClass);
		console.log(elementsByTag);
		</script>
		</body>
		</html>

Método textContent()	Ele pode modificar ou alterar o conteúdo de texto ou HTML dos elementos.	<!DOCTYPE html> <html> <head> <title>textContent Example</title> </head> <body> This is some text. <script> const paragraph = document.getElementById('my-paragraph'); paragraph.textContent = 'This is updated text.'; </script> </body> </html>

Método setAttribute()	É utilizado para alterar os atributos (por exemplo, src, href, class, id) dos elementos, o que pode afetar seu comportamento ou aparência.	<!DOCTYPE html>
		<html>
		<head>
		<title>setAttribute Example</title>
		</head>
		<body>
		<img id="my-image" src="your-old-image.jpg">
		<script>
		const image = document.getElementById('my-image');
		image.setAttribute('src', 'your-new-image.jpg');
		</script>
		</body>
		</html>

Adicionando Elementos	Adicionando dinamicamente novos elementos à página com base em interações do usuário ou outras condições.	<!DOCTYPE html>
		<html>
		<head>
		<title>createElement Example</title>
		</head>
		<body>
		<ul id="my-list">
		 - Item 1
		 - Item 2
		
		<script>
		const list = document.getElementById('my-list');
		const newItem = document.createElement('li');
		newItem.textContent = 'Item 3';
		list.appendChild(newItem);
		</script>
		</body>
		</html>

Método cloneNode()	Criando cópias de elementos existentes que podem ser inseridos em outros locais do documento.	`<!DOCTYPE html>`
		`<html>`
		`<head>`
		`<title>createElement Example</title>`
		`</head>`
		`<body>`
		`<ul id="my-list">`
		`<li>Item 1</li>`
		`<li>Item 2</li>`
		`</ul>`
		`<script>`
		`const list = document.getElementById('my-list');`
		`const firstItem = list.querySelector('li');`
		`const clonedItem = firstItem.cloneNode(true);`
		`list.appendChild(clonedItem);`
		`</script>`
		`</body>`
		`</html>`

objeto window	O objeto global window representa a janela ou aba do navegador e serve como a raiz do BOM.	`window.alert(message): Displays a simple alert dialog with the specified message.`
		`window.confirm(message): Shows a confirmation dialog with "OK" and "Cancel" buttons and re`
		`window.open(url, name, specs, replace): Opens a new browser window or tab.`
		`window.close(): Closes the current window or tab.`
		`window.location: Provides information about the current URL and allows navigation.`
		`window.setTimeout(function, delay): Executes a function after a specified delay.`
		`window.localStorage and window.sessionStorage: Allow data storage on the client side.`
		`window.history: Provides access to the browser's session history.`

Objeto navigator	O objeto navigator fornece informações sobre o navegador do cliente, como o nome do navegador, versão e recursos suportados.	`const browserName = navigator.appName;` `const browserVersion = navigator.appVersion;`
objeto screen	O objeto screen fornece detalhes sobre a tela do usuário, incluindo suas dimensões e profundidade de cor.	`const screenWidth = screen.width;` `const screenHeight = screen.height;`

objeto history	O objeto history representa o histórico de sessão do navegador, permitindo que você navegue para trás e para frente no histórico de navegação do usuário.	`history.back(); // Navigates back one page`
		`history.forward(); // Navigates forward one page`

objeto location	O objeto location fornece informações sobre a URL atual e permite que você manipule a URL, redirecionando o usuário para outras páginas da web.	`const currentURL = location.href;` `location.href = 'https://example.com'; // Redirects the user to a new URL`

Exemplo de BOM	Isso representa o exemplo combinado dos métodos de BOM acima.	<!DOCTYPE html>
		<html>
		<head>
		<title>BOM Example</title>
		</head>
		<body>
		<button id="alertButton">Show Alert</button>
		<button id="openWindowButton">Open Window</button>
		<button id="navigateBackButton">Go Back</button>
		<button id="changeURLButton">Change URL</button>
		<script>
		// Access HTML elements
		const alertButton = document.getElementById('alertButton');
		const openWindowButton = document.getElementById('openWindowButton');
		const navigateBackButton = document.getElementById('navigateBackButton');
		const changeURLButton = document.getElementById('changeURLButton');
		// Attach event listeners
		alertButton.addEventListener('click', () => {
		window.alert('Hello, this is an alert!');
		});
		openWindowButton.addEventListener('click', () => {
		window.open('https://example.com', '_blank');
		});
		navigateBackButton.addEventListener('click', () => {
		history.back(); // Navigates back one page in the user's browsing history.
		});
		changeURLButton.addEventListener('click', () => {
		location.href = 'https://example.com'; // Redirects the user to a new URL.
		});
		</script>
		</body>
		</html>

firstElementChild() e lastElementChild()	Utiliza as propriedades firstElementChild e lastElementChild para acessar os primeiros e últimos nós filhos de qualquer elemento.	<!DOCTYPE html>
		<html>
		<head>
		<title>DOM Traversing Example</title>
		</head>
		<body>
		
		 Child 1
		 Child 2
		
		<script>
		const parent = document.getElementById("parent");
		const firstChild = parent.firstElementChild;
		const lastChild = parent.lastElementChild;
		console.log(firstChild.textContent); // Outputs: "Child 1"
		console.log(lastChild.textContent); // Outputs: "Child 2"
		</script>
		</body>
		</html>

Elemento container	Para encontrar elementos dentro de um container, você geralmente usa métodos que permitem consultar elementos com base em vários critérios, como nome da tag, classe ou outros atributos.	<!DOCTYPE html>
		<html>
		<head>
		<title>DOM Traversing Example</title>
		</head>
		<body>
		
		 Paragraph 1
		 Paragraph 2
		 Paragraph 3
		
		<script>
		const container = document.getElementById("container");
		const singleElement = container.querySelector(".myClass");
		const multipleElements = container.querySelectorAll(".myClass");
		console.log(singleElement.textContent); // Outputs: "Paragraph 1"
		console.log(multipleElements[1].textContent); // Outputs: "Paragraph 2
		</script>
		</body>
		</html>

element.style.property = value	Uma maneira de acessar e modificar os estilos inline de um elemento HTML usando a propriedade style.	<!DOCTYPE html>
		<html>
		<head>
		  <title>DOM Styling Example</title>
		</head>
		<body>
		  <button id="myButton">Click Me</button>
		  <script>
		  const button = document.getElementById("myButton");
		  button.style.backgroundColor = "blue";
		  button.style.color = "white";
		  button.style.fontSize = "16px";
		  </script>
		</body>
		</html>

element.classList	Você pode usar a propriedade classList para adicionar, remover ou alternar classes CSS em um elemento.	<!DOCTYPE html>
		<html>
		<head>
		  <title>DOM Styling Example</title>
		</head>
		<body>
		  This is a div
		  <button id="myButton">Toggle Class</button>
		  <script>
		  const div = document.getElementById("myDiv");
		  const button = document.getElementById("myButton");
		  function toggleClassAndColor() {
		  div.classList.toggle("active");
		  div.classList.toggle("inactive");
		  // Check if the "active" class is present and change the background color acc
		  if (div.classList.contains("active")) {
		  div.style.backgroundColor = "blue";
		  } else {
		  div.style.backgroundColor = "red";
		  }
		  }
		  button.addEventListener("click", toggleClassAndColor);
		  </script>
		</body>
		</html>

element.setAttribute	Um método para usar o método setAttribute para definir ou modificar o atributo de estilo de um elemento, que é uma string contendo CSS inline.	<!DOCTYPE html>
		<html>
		<head>
		  <title>DOM Styling Example</title>
		</head>
		<body>
		  This is a red paragraph.
		  <button id="btn">Click to change Color of above paragraph</button>
		  <script>
		  const paragraph = document.getElementById("myParagraph");
		  const btn=document.getElementById('btn');
		  btn.addEventListener('click',()=>{
		  paragraph.setAttribute("style", "color: blue; font-size: 18px;");
		  })
		  </script>
		</body>
		</html>

element.style.cssText	A propriedade cssText permite que você defina todo o estilo inline de um	<!DOCTYPE html> <html> <head> <title>DOM Styling Example</title> </head>

	elemento como uma string.	<body>
		  This is a paragraph.
		  <button id="btn">Click to change Color and bold</button>
		  <script>
		  const text = document.getElementById("myText");
		  const btn=document.getElementById('btn');
		  btn.addEventListener('click',()=>{
		  text.style.cssText = "color: red; font-weight: bold;";
		  })
		  </script>
		</body>
		</html>

element.style.setProperty	Este método permite definir uma propriedade CSS específica com uma prioridade opcional para o estilo inline de um elemento.	<!DOCTYPE html>
		<html>
		<head>
		  <title>DOM Styling Example</title>
		</head>
		<body>
		  <h1 id="myHeading">This is a heading.</h1>
		  <button id="btn">Click Here</button>
		  <script>
		  const heading = document.getElementById("myHeading");
		  const btn=document.getElementById('btn');
		  btn.addEventListener('click',()=>{
		  heading.style.setProperty("color", "violet", "important");
		  })
		  </script>
		</body>
		</html>

element.style.removeProperty	Você pode usar o método removeProperty para remover uma propriedade CSS específica do estilo inline de um elemento.	<!DOCTYPE html>
		<html>
		<head>
		  <title>DOM Styling Example</title>
		</head>
		<body>
		  This is a styled paragraph.
		  <button id="btn">Click Here</button>
		  <script>
		  const paragraph = document.getElementById("myParagraph");
		  const btn=document.getElementById('btn');
		  btn.addEventListener('click',()=>{
		  paragraph.style.removeProperty("color");
		  })
		  </script>
		</body>
		</html>

Skills Network